TokenWave

SWE-CoWork: Evaluating Coding Agents in Organisations of Employee Digital Twins

TokenWave AI

A future software organisation could be led by humans who set direction and staffed by agent programmers who coordinate and deliver the work. We propose a paradigm shift in software-engineering evaluation towards this future: from giving an agent a packaged coding task to placing it in an organisation of employee digital twins and asking it to own the work. We introduce SWE-CoWork, which instantiates this setting through 101 development assignments across 101 real repositories, 11 languages, and 12 domains. Each organisation contains 26 modelled colleagues, yielding 2,626 employee profiles across the benchmark. These are functional digital twins of employee roles: they have distinct responsibilities, bounded local knowledge, and controlled collaboration behaviour. Starting from a brief handoff, the agent must establish what to build by consulting the right colleagues, following up on unresolved questions, and reconciling their answers with project boards, versioned documents, chat, reviews, and CI history. It must then implement and validate the change. A median task requires 2,955 production lines across 60 files and is evaluated by 32 executable tests; every task retains a fixed fail-to-pass and pass-to-pass contract. We evaluate six models on 101 tasks using F2P, P2P and Pass rate, compare Seed 2.1 Pro with a complete Oracle-Spec, and analyse Opus-5.5 trajectories. Opus-5.5 passes 63.84% of F2P nodes and 93.24% of P2P nodes; 6.93% of tasks pass both test classes completely. Providing the complete specification raises Seed 2.1 Pro's F2P from 21.01% to 55.76%. The Opus-5.5 trajectory analysis locates unresolved work in requirements that never leave a colleague and requirements received but implemented incorrectly. SWE-CoWork makes working with a population of employees part of the evaluation of a digital software engineer.

Date: October 11, 2026
Website: https://tokenwave.ai/

1 Introduction

Imagine a software organisation in which humans set the direction and a team of agent programmers carries out the engineering. A human leader chooses what to build, sets priorities, and resolves major trade-offs. The agents take ownership of the resulting work: they break down objectives, recover missing context, coordinate dependencies, implement changes, and verify delivery. One person could lead many such digital engineers, assigning outcomes and judging progress while the agents manage the everyday details of execution. This is a vision of software development organised around teams of digital employees.

An agent in such a team must be able to inherit work already in progress, understand decisions made by others, and carry an assignment through implementation, verification, and handoff with limited supervision. A request to a *digital software engineer* may be as brief as "finish the 2026.09 upgrade; the previous owner is on leave." The request leaves much of the specification unstated. Establishing what the work actually requires, and coordinating with the people who know, is part of the responsibility being delegated.

A paradigm shift: from coding tasks to organisations of digital twins. We introduce SWE-CoWork around a proposed paradigm shift in evaluation: give the agent an engineering organisation in which to work,

with the people who hold its knowledge represented as *employee digital twins*. We use this term for functional models of employees' roles, responsibilities, local knowledge, and collaboration behaviour in an authored workplace. Each SWE-CoWork organisation contains 26 such colleagues; across 101 organisations, the benchmark contains 2,626 employee profiles. The evaluation asks whether an agent can enter this organisation, establish what an assignment requires through its people and records, and take responsibility for the outcome. This shifts the focus from executing a supplied specification to owning work within a team. It evaluates a prerequisite for the future above: an individual agent's ability to take responsibility for an assignment while working with many colleagues.

This reflects how software engineering is done in practice. Most engineering work begins from a state produced by other people rather than from a blank slate. Developers spend substantial effort understanding existing systems, reconstructing earlier decisions, and locating the people who hold missing knowledge (Ko et al., 2007; LaToza et al., 2006; Sillito et al., 2008; Xia et al., 2018). The relevant state is distributed across code, documents, discussions, reviews, CI runs, and people's local knowledge, and these sources do not always agree. An old design document may describe a superseded approach. A green CI run may belong to the wrong revision. One colleague may know a compatibility constraint but not the downstream dependency it affects. A trace recovered midway through implementation may overturn the initial diagnosis. A strong engineer handles these conditions as part of ordinary work, interleaving investigation, conversation, implementation, and verification until both the code and the understanding behind it converge.

Existing coding benchmarks largely begin after this process has already been compressed into a task. The SWE-bench lineage (Jimenez et al., 2024), DeepSWE (Huang et al., 2026), SWE Atlas (Raghavendra et al., 2026), FrontierSWE (Kondra et al., 2026), CursorBench (Cursor, 2026), and Real-SWE (Das et al., 2026) evaluate increasingly realistic and long-horizon changes in substantial repositories. Work on underspecification and clarification (Kim et al., 2026; Pu et al., 2026; Edwards and Schuster, 2026) withholds information from the request and studies whether an agent can recover it, often through interaction with a single simulated user. These settings are valuable, but they still largely assume that the unit of work has already been isolated for the agent. They ask whether an agent can solve a software task once that task has been packaged. A digital engineer faces a broader problem: it must enter work already in progress, reconstruct the state of that work from the surrounding organization, determine what remains valid or unresolved, and carry the work forward.

Each SWE-CoWork task couples a real open-source repository with an organisation of employee digital twins and asks the agent to deliver an ongoing piece of development work. Figure 1 illustrates the distinction on a single Click requirement. In an issue-style setting, the requirement is presented directly to the agent. In SWE-CoWork, the same executable requirement is embedded inside a broader assignment: the agent must determine how it fits the current project state, recover the relevant prior decisions, consult the right owner when necessary, implement the change, and revise its understanding when new evidence arrives. The underlying requirement and its tests are identical in both cases; what changes is whether the surrounding engineering work has already been reduced to a specification.

Many employees, distributed responsibility. The employee population is central to this shift. A typical task distributes colleague-held requirements across six accountable owners within its 26-person organisation. A module maintainer, a compatibility liaison, and a release coordinator hold different parts of the engineering context; their answers must be connected to each other and to the repository. The agent must learn whom to approach, supply the context that person needs, follow a referral, and return when an answer is deferred. Shared documents and chat preserve the team's history, including conclusions that later changed. Together, the employee twins and these records represent the organisation through which the work can be understood. Keeping ownership and knowledge attached to distinct employees makes finding, coordinating, and following through with people part of the engineering task.

The job. SWE-CoWork contains 101 tasks from 101 upstream repositories across 11 languages and 12 application domains. Each task composes a sequence of upstream changes into one cumulative development objective: a median task adds 2,955 production lines across 60 files and is graded by 32 verifier test nodes. The agent receives only a short onboarding note, together with a frozen repository and access to the surrounding workplace: a project board, versioned documents, chat, a staff directory, merge-request history, and CI records. Across the benchmark, these organisations contain 40,836 chat messages. Knowledge is partitioned across

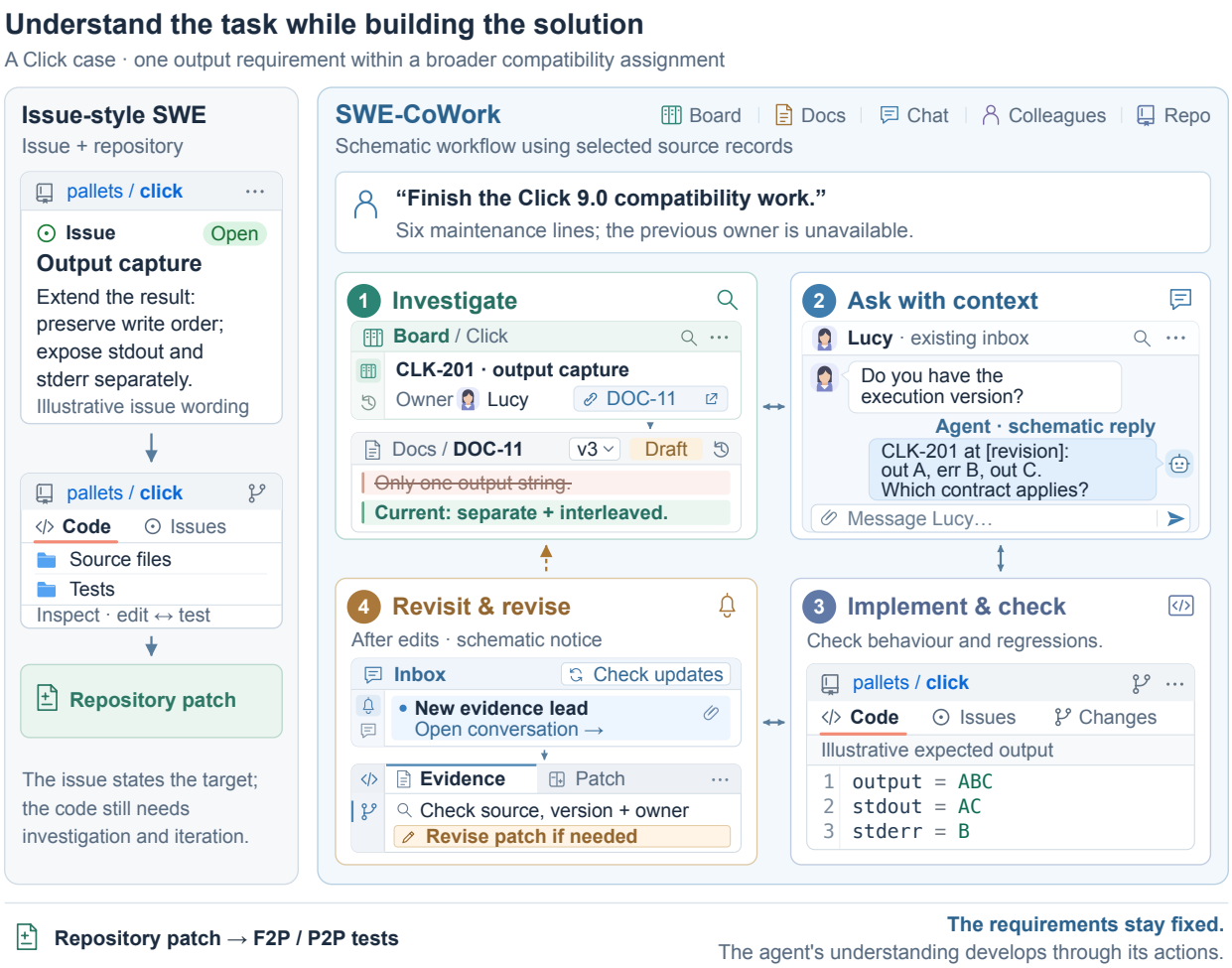


Figure 1 From a packaged task to work inside an organisation: the same Click requirement as an issue-style task (left) and as a SWE-CoWork assignment (right). In SWE-CoWork, employee digital twins hold distinct parts of the task knowledge. The agent reconstructs the state of the work, consults the appropriate owner, implements and verifies the change, and revises its interpretation when new evidence arrives. Interfaces are schematic; Appendix D gives the source records.

colleagues; records preserve outdated conclusions alongside current ones; execution evidence must be matched to the revision that produced it; and relevant information can surface only after implementation has begun (Figure 4). The agent must decide what to read, whom to ask, which evidence to trust, what to build, and how to verify its work. Finally, it must produce a handoff document summarizing the completed changes, verification results, and any remaining issues.

Solvable, and graded by code. The workplace is complex but not arbitrary. Every scored requirement has a designed route to discovery through workplace records, executable evidence, or the colleague who owns the relevant knowledge. At the same time, the target never changes: each task is graded by a fixed fail-to-pass and pass-to-pass contract defined independently of the agent's trajectory. Difficulty therefore comes from operating correctly inside the engineering context—recovering, confirming, and acting on the right information—while success is ultimately decided by executable behavior.

Separating engineering context from implementation. For every SWE-CoWork task, we construct a paired Oracle-Spec condition that provides the complete requirements up front while keeping the repository, tests, scaffold, and interaction budget unchanged. This creates two views of the same engineering objective: one in which the agent must reconstruct and act on organizational context, and one in which that context has already been compiled into a specification. Under this matched design, the performance gap measures

the added burden of establishing requirements within the same execution budget. Runtime logs of searches, document reads, questions, referrals, CI inspection, and code changes then let us locate where each missed requirement was lost: never discovered, discovered but not validated, disclosed but not incorporated, or understood but implemented incorrectly.

Outcomes and representative trajectories. We evaluate six models on 101 tasks using F2P, P2P and Pass rate (Table 3). Opus-5.5 passes 63.84% of F2P nodes and 93.24% of P2P nodes, with 6.93% of tasks passing both test classes completely. The Oracle comparison focuses on Seed 2.1 Pro (Table 4): supplying the complete requirements raises its pooled F2P from 21.01% to 55.76%.

The Opus-5.5 trajectory analysis divides unresolved linked tests between requirements that stayed with a colleague (42%) and requirements received but implemented incorrectly (56%). The traces also expose incomplete follow-up and delivery notes that leave failed requirements unacknowledged. This study connects final outcomes to how an agent establishes, implements and reports its work inside the team.

Contributions.

- **Engineering organisations built from employee digital twins.** SWE-CoWork provides 101 substantial development assignments across 101 repositories, 11 languages, and 12 domains. Each organisation contains 26 modelled colleagues with distinct responsibilities and bounded knowledge, together with a shared workplace history and a fixed executable contract.
- **A paired evaluation of the shift from tasks to work ownership.** The paired CoWork and Oracle-Spec design distinguishes establishing requirements through an organisation of employees from implementing a fully specified objective; the empirical comparison focuses on Seed 2.1 Pro.
- **Trajectory-level diagnosis of how work breaks down.** Runtime traces connect unresolved tests to their requirements and to the stages at which those requirements were lost, turning a final code score into an account of how the agent investigated, coordinated, interpreted, and executed the work. Applied to Opus-5.5, the diagnosis identifies distinct failures in obtaining requirements, implementing them, following up with colleagues, and reporting unfinished work.

2 Related Work

From code changes to realistic engineering work. Repository-level coding benchmarks have steadily expanded the scope of what an agent is asked to execute. DeepSWE (Huang et al., 2026; Datacurve, 2026) evaluates original tasks in real repositories; SWE Atlas (Raghavendra et al., 2026) covers codebase questions, test writing, refactoring, and implementation; FrontierSWE v2 (Kondra et al., 2026) studies implementation, optimisation, and research tasks under long interaction budgets; CursorBench 4.0 (Cursor, 2026; Jain, 2026) derives tasks from real developer activity; and Real-SWE (Das et al., 2026) extends evaluation to private production codebases and business tooling. Together, these benchmarks substantially increase repository realism, task scale, and execution horizon. Their unit of evaluation, however, is still primarily a development task that has already been identified and presented to the agent. SWE-CoWork studies the preceding part of engineering work: entering an ongoing project, reconstructing what the work currently requires from its surrounding organizational state, and then carrying that work through to an executable outcome.

Underspecification, clarification, and requirement recovery. A complementary line of work relaxes the assumption that all relevant information appears in the initial request. RealSWE (Kim et al., 2026) varies how much information is exposed for a fixed task, while LHAW (Pu et al., 2026) removes information along goal, constraint, input, and context dimensions. *Ask or Assume* (Edwards and Schuster, 2026) and Dialogue SWE-Bench (King and Flanigan, 2026) introduce interactive users who can clarify an issue, and ICAE-Bench (Peng et al., 2026) grounds user responses in withheld product constraints. τ^{τ}-Bench (Shi et al., 2026) further requires a developer agent to recover requirements from business artifacts and interaction with a client. These settings establish that successful coding may require information acquisition rather than direct execution of a complete prompt.

Benchmark	Engineering task	Initial information	Information sources	Information process	Evaluation
DeepSWE 1.1 (Huang et al., 2026; Datacurve, 2026)	Repo change	Request Underspecified	Repo	n/r	Patch tests
SWE Atlas (Raghavendra et al., 2026)	Code Q&A Test writing Refactor	Request Underspecified	Repo	No live Q&A	Execution Rubric
FrontierSWE v2 (Kondra et al., 2026)	Build Optimise Research	Brief	Workspace Self-check	n/r	Task score
CursorBench 4.0 (Cursor, 2026; Jain, 2026)	Edit Refactor Investigate	Request Underspecified	Code Tools	n/r	Agentic grader
Real-SWE (Specific Labs) (Das et al., 2026)	Enterprise change	Brief Underspecified	Repo Docs Board Chat	n/r	n/r
RealSWE (paper v2) (Kim et al., 2026)	Bug fix Feature	Issue variants Underspecified	Repo	Static request No live Q&A	Fixed target
τ^{τ}-Bench (Shi et al., 2026)	Agent build	Underspecified	Docs APIs Code	Client Q&A Conflicting history Superseded records	User simulation
SWE-CoWork (ours)	Multi-PR Repo change	Onboarding brief Underspecified	Repo Board Docs Chat MR/CI	Multi-owner Q&A Versioned records Conflicting history Execution-time leads	F2P/P2P Fixed contract

task initial information information source interaction / dynamics evaluation

n/r = not reported in the cited public description.

Table 1 How recent software-engineering benchmarks expose engineering work to an agent. *Initial information* denotes what is provided at the outset; *information sources* denote where task-relevant evidence resides; and *information process* captures whether the agent must clarify, reconcile, or respond to information that changes during execution. "n/r" indicates that the property is not described in the cited public material.

SWE-CoWork builds on this direction by modelling the employees among whom requirements are distributed. Its population of employee digital twins gives each source of local knowledge an identity, a responsibility, and a bounded interaction policy. Relevant evidence is distributed across these colleagues, project records, repository history, and execution traces. Sources may disagree because they reflect different revisions or points in time; resolving one question may reveal another dependency; and evidence relevant to an earlier decision can surface after implementation has already begun. The resulting problem is therefore not only whether the agent knows to ask for missing information, but whether it can determine where authoritative information should come from, reconcile it with the current project state, and update its work accordingly.

Agents in workplace environments. Several benchmarks place agents inside broader organizational or computer-use environments rather than exposing a single application or repository. TheAgentCompany (Xu et al., 2025), for example, evaluates professional tasks in a simulated company containing colleagues and workplace services. Such environments establish the importance of navigating organizational artifacts and interacting with other actors. Our proposed paradigm builds on this view of an organisation as the evaluation environment. SWE-CoWork specialises it to substantial software-engineering work by coupling a population of employee digital twins to a real repository and its development history. The task requires repository-scale implementation, and final correctness is measured by a fixed fail-to-pass and pass-to-pass executable contract rather than only by process checkpoints.

Interaction fidelity is particularly important in this setting. Fully LLM-simulated users can introduce run-to-

run variation in what information is revealed (Seshadri et al., 2026). In SWE-CoWork, the environment determines which facts a colleague is permitted to disclose and under what conditions, while a language model realizes the response in natural language. Each employee twin therefore combines a defined responsibility and knowledge boundary with a conversational interface. The environment fixes the disclosure rules and task contract across runs, while the wording of a reply can vary.

Diagnosing agent behavior beyond final correctness. Prior work has increasingly examined the trajectories that produce a final coding outcome. SWE-RPG (Zhou et al., 2026) provides representations for studying planning and clarification behavior in software-agent trajectories, while τ-bench (Yao et al., 2024) highlights reliability across repeated interactive executions. Coding-agent frameworks such as SWE-agent, OpenHands, and Agentless (Yang et al., 2024; Wang et al., 2025; Xia et al., 2024) further show that agent scaffolding can materially shape observed capability. SWE-CoWork therefore records not only the submitted patch but also the path by which the agent established the task: searches, document reads, questions, referrals, execution evidence, and code changes. These traces allow missed requirements to be localized to distinct stages—for example, evidence that was never discovered, information that was obtained but not incorporated, or a requirement that was understood but implemented incorrectly—and support comparisons across both models and scaffolds.

Table 1 summarizes the closest settings along the dimensions most relevant to this distinction. Appendix A provides a broader view of repository-level, interactive, and workplace-agent benchmarks.

3 Engineering Organisations of Employee Digital Twins

Each SWE-CoWork task places a coding agent in an engineering organisation consisting of a real software repository, a shared workplace, and a population of employee digital twins. Starting from a brief handoff rather than a complete specification, the agent must reconstruct the requirements through workplace records and interactions with colleagues, then implement, verify, and deliver the required changes. Each task has a fixed executable contract, evaluated by fail-to-pass (F2P) and pass-to-pass (P2P) tests, allowing us to assess the agent's ability to work within an organisation while keeping the underlying software objective unchanged.

3.1 Employee Digital Twins

Around each repository we construct an organisation of 26 modelled colleagues, giving 2,626 employee profiles across the 101 tasks. Each twin is a functional representation of an employee in this authored organisation: a named person with a role, an area of responsibility, a boundary on what they know, and rules for how they respond to the agent. Responsibilities and knowledge are grounded in the repository's files, declarations, and modules. The population spans one target project and four neighbouring projects, which provide organisational context while remaining outside the assignment's scope. Within this population, a median of six employees own the colleague-held requirements that determine the task's executable outcome (Table 7).

What an employee twin represents. Three aspects make these colleagues part of the engineering environment. *Responsibility* identifies the person accountable for a module or decision. *Local knowledge* determines which requirements that person can explain and which questions require a referral. *Collaboration behaviour* governs the exchange: an owner can ask for a reproduction or caller path, answer one topic at a time, or defer an answer that the agent must follow up. The environment controls what information may be disclosed at each step; a language model expresses the response in the employee's voice. The twin therefore links a conversational partner to a specific part of the software and to a controlled source of task knowledge.

From individual twins to an organisation. The twins share project records that connect people to the work: board cards name owners, documents record decisions, and chat and review threads preserve earlier exchanges. An answer from one employee may need to be reconciled with another employee's constraint or with a newer repository revision. The agent must track which owner can resolve each open question and carry those conversations through as implementation proceeds. This is the organisational structure behind

the proposed paradigm shift: the specification must be established through a population with distributed knowledge and responsibility.

Through a dedicated tool interface, the agent can search a staff directory, inspect the project board, open versioned documents, search and read chat, message colleagues, inspect merge requests and CI records, and check for newly available activity. The agent accesses employee twins through the same workplace in which it finds their decisions and work history.

Workplace records are written in Chinese, reflecting the team that produced them, while repository code and executable contracts are primarily in English. The setting therefore also preserves the multilingual mixture common in software teams whose internal communication and codebase use different languages.

The handoff. The agent enters work that is already in progress rather than receiving a self-contained issue. It begins with a short onboarding note that identifies the development objective and points to the workplace surfaces from which the current state can be reconstructed. The note also establishes task-wide boundaries on changes to existing tests, excludes neighbouring projects from scope, and identifies historical terminology that may need to be reconciled with the current repository. It does not enumerate the complete acceptance criteria.

From that point onward, the agent decides how to establish the work. It may inspect records, follow references across artifacts, compare evidence against the current repository state, or contact the colleague responsible for a particular decision. At completion, it produces a delivery note recording the requirements it established, the changes it made, the validation it performed, and any questions that remain unresolved.

3.2 Composition

SWE-CoWork contains 101 tasks drawn from 101 upstream repositories across 11 programming languages and 12 application domains, ranging from web frameworks and compilers to games, payments, and scientific software (Figure 2; Appendix B). 11 tasks involve substantive changes in more than one programming language.

The underlying engineering work is substantial. The median reference solution adds 2,955 production lines across 60 files, excluding tests, documentation, configuration, fixtures, and generated artifacts. Each task is therefore large enough to require repository-level reasoning rather than a localized edit. Alongside the workplace version of every task, we maintain a complete Oracle-Spec document that enumerates the requirements encoded by the same executable contract. Across the 101 tasks, that contract contains 2,699 F2P nodes and 562 P2P nodes, for 3,261 executable checks in total.

3.3 Shared Records and Organisational History

The workplace surrounds each assignment with records from a wider organisation. Across the benchmark release, the organisations contain 10,329 board cards, 5,097 versioned documents, 40,836 chat messages, 1,194 merge requests, and 2,626 colleague profiles. The initial onboarding note contains a median of 206 words, whereas the authored document corpus contains a median of 5,453 words. The document collection therefore offers substantially more context than the handoff, distributed across versions and topics.

Only a small fraction of this material is relevant to any particular requirement. The agent must therefore allocate attention: which artifact to inspect, which historical claim still applies, which execution record matches the current revision, and which colleague is likely to hold the missing piece of context. Choosing where to look is part of the task rather than a preprocessing step. We keep the overall organisation size roughly comparable across tasks so that differences in difficulty arise primarily from the software work and the structure and placement of relevant evidence, rather than from simply giving some tasks much larger workplaces.

History, not a snapshot. The workplace represents an evolving project rather than a clean snapshot of its final truth. Versioned documents, board histories, review threads, CI records, and conversations retain intermediate diagnoses, superseded plans, rejected approaches, and observations tied to particular revisions. Names of interfaces, modules, or protocols may differ between historical records and the current codebase. A record is therefore evidence to interpret, not necessarily a fact to copy.

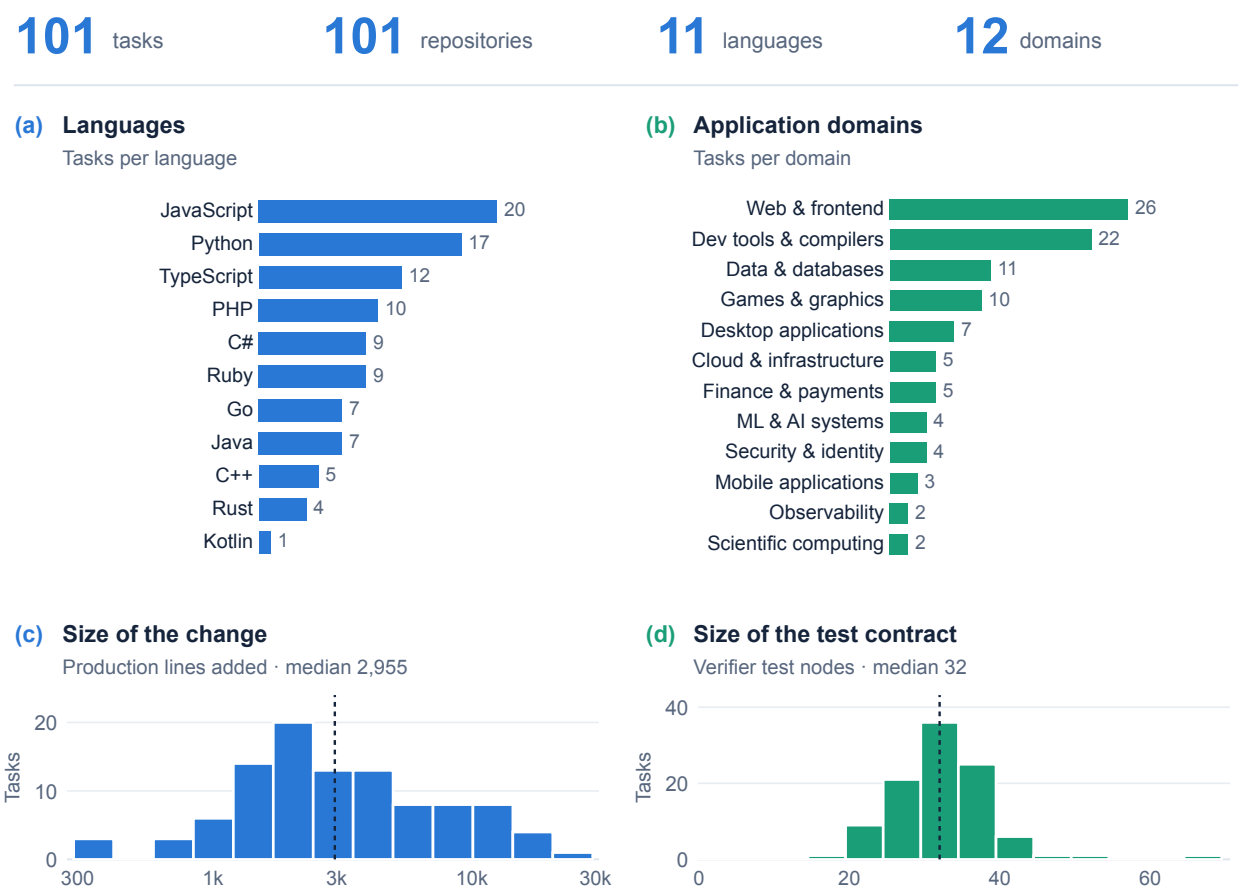


Figure 2 Composition of the 101 SWE-CoWork tasks. (a) Primary language of the production changes. (b) Application domain of the upstream repository. (c) Production lines added by the reference changes, shown on a logarithmic axis. (d) Verifier test nodes per task. Dashed lines mark medians.

Access is similarly bounded. Private groups and direct messages that do not include the agent are inaccessible, while neighbouring projects remain visible enough to create realistic context but are explicitly out of scope. The agent must work within the information boundaries of the organisation rather than assuming unrestricted access to a hidden global state.

3.4 Scoring

We report three outcome metrics: **F2P**, the proportion of fail-to-pass verifier nodes satisfied by the submitted implementation; **P2P**, the proportion of pass-to-pass nodes preserved; and **Pass rate**, the proportion of tasks for which every F2P and every P2P node passes in the same run. F2P and P2P pool passed and total nodes across tasks.

Each task contains multiple independently testable behaviours. F2P therefore measures partial delivery, while P2P records preservation of existing functionality. Pass rate measures complete delivery of the contract. Section 5 gives the aggregation formulas and distinguishes these primary scores from the trajectory diagnostics.

4 Working a SWE-CoWork Task

A SWE-CoWork task puts the agent to work alongside the employee digital twins defined in Section 3.1. It rewards the habits of a good engineer joining a team: read the handoff, find the evidence, ask the right person a precise question, build, check, and change course when the facts change. This section follows that loop,

shows how an employee's local knowledge changes a real patch, and describes the situations the organisation creates.

4.1 Working with the Employee Twins

The environment does not prescribe an order. An agent can return to any step when new evidence changes its picture of the work.

1. **Orient.** Read the onboarding note, explore the repository, and establish the target project, scope and first leads.
2. **Investigate.** Search the board, documents, chat and history for the behaviour the work requires, reconciling renamed interfaces, versions and the provenance of each record.
3. **Ask.** Find the colleague who owns an open decision and bring them a question with enough context to answer. Follow referrals; answer their follow-up questions.
4. **Build and verify.** Change the repository, keep existing tests and neighbouring projects intact, and run the tests.
5. **Revise.** Read new messages, take a late trace or correction back to its owner, and update the implementation when the evidence demands it.
6. **Deliver.** Record the confirmed requirements, changes and checks in the delivery note.

Every scored requirement has a route to discovery through workplace records or the employee who owns the relevant knowledge. Multiple conversations may remain open while the agent builds: one owner is waiting for a reproduction, another has referred a question, and a third has supplied a constraint that changes the patch. The agent must retain the connection between each question, its owner, and the affected code. Disclosure follows the environment's rules for that owner and conversation state, while the language model supplies the wording. This preserves the knowledge boundaries across runs while allowing natural dialogue.

4.2 Inside a run: one preference changes the patch

Task 094 hands the agent the 2026.09 cumulative upgrade of Kimai, a PHP time-tracking application: 21 feature cards, 51 documents, 26 employee digital twins, and an owner who is away. Over three hours and 22 minutes, a frontier agent made about 1,000 tool calls, read 16 documents, reviewed 41 board cards, and sent 19 messages to six colleagues. One requirement shows how an employee twin's local knowledge affects the delivered software (Figure 3).

Card KIM-210 asks for decimal-hour durations in timesheet exports. The arithmetic is one division. The agent found the spreadsheet renderer, added a column option for a decimal base, and wrote a test configuring 3,600 seconds per unit: a 5,400-second entry exported as 1.5 hours, and twelve focused tests passed with 74 assertions. These checks validated the arithmetic while leaving the user workflow unresolved.

The design document left open how a user turns the feature on, so the agent took its working result to the export owner:

> "My XLSX cell and total calculate to 1.5. Please confirm whether that is the intended decimal behavior…" — *agent*

> "…when the current user's `timesheet.export_decimal` preference is true, the value is 1.5 hours… The preference is evaluated on every export, even when one renderer is reused for different users." — *export owner*

That answer changed the code. The renderer now reads the current user's preference on every render, and the revised test covers zero durations, durations over 24 hours, totals, and one renderer reused across two users. A column that could display decimals became a feature users could actually enable—and a preference read once would have leaked one user's choice into another user's export. The same pattern ran across the task: 17 of 21 new behaviours delivered and all 15 existing behaviours preserved.

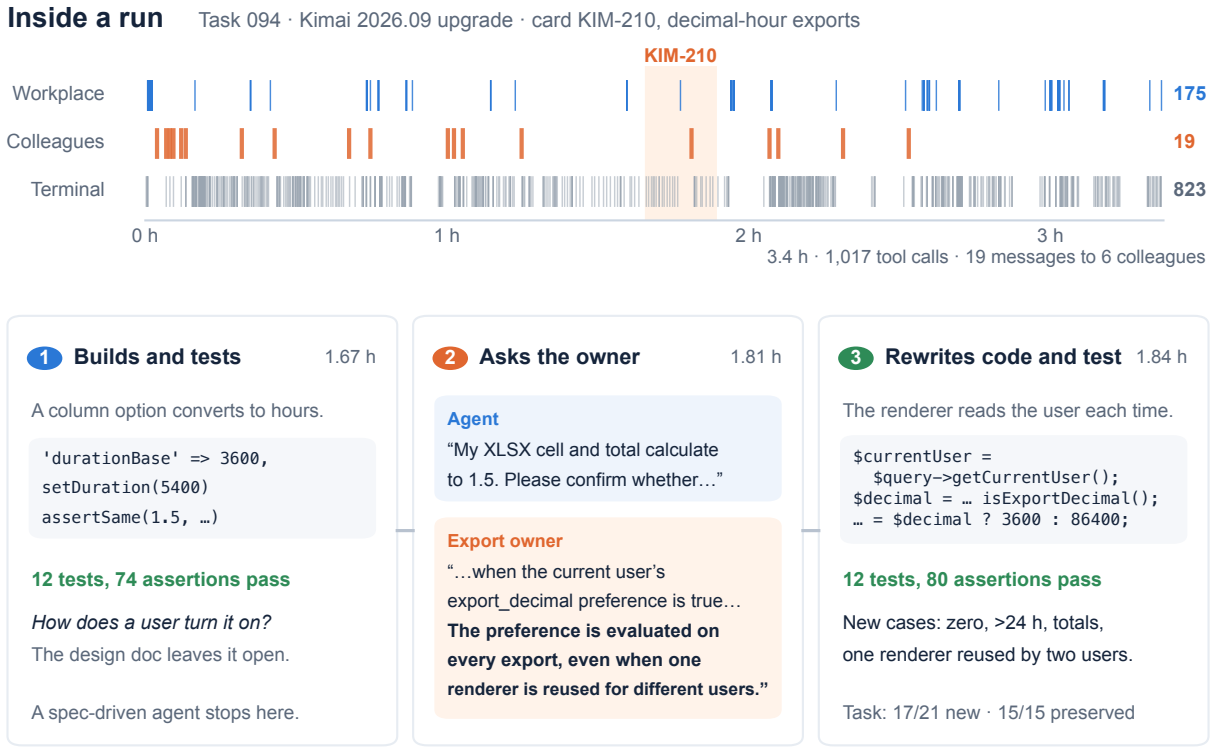


Figure 3 One requirement in a recorded SWE-CoWork run (task 094, Kimai). The agent's first implementation passes its own tests but leaves the user workflow open. A question to the owner, carrying the observed result, establishes that the current user's preference controls the export and is read on every render. The patch and its test change accordingly. Times are elapsed hours from the start of the run; quotations are from the recorded exchange. This is a pilot run on an earlier package revision, separate from the cohort of Section 6.1.

4.3 What the organisation throws at the agent

Real engineering information is scattered, partial and sometimes wrong. SWE-CoWork builds these situations into every organisation (Figure 4, Table 2).

Obstacle	Authored situation	Engineering judgment
Late evidence	A trace, version correction or reproduction lead arrives after implementation begins.	Check updates, follow the notice to its owner and reassess the affected behaviour.
Superseded conclusions	An old summary or colleague hypothesis remains visible alongside a later correction.	Reconstruct the timeline and establish which decision or observation still applies.
Ambiguous provenance	A partial log omits context, carries an older run ID, or mixes result output with cleanup warnings.	Align the input, run and revision; separate observations from inferred causes.
Fragmented ownership	A contact knows an adjacent boundary, refers the agent elsewhere, or needs further context before answering.	Find the accountable owner and complete clarification for each unresolved topic.
Scope distractors	Cancelled or duplicate cards, rejected approaches and neighbouring projects remain searchable.	Use status histories, project membership and review outcomes to recover the current scope.

Table 2 Workplace situations and the engineering judgment each one exercises.

Evidence that arrives mid-build. As the agent changes the repository, new messages appear: a recovered trace, a corrected version label, a reply from another team. Each points to a colleague who owns the answer. A notice can challenge an interpretation already written into code, and the agent must decide whether to revisit it. Owners can be consulted at any time, so an agent that asks the right question early never needs to

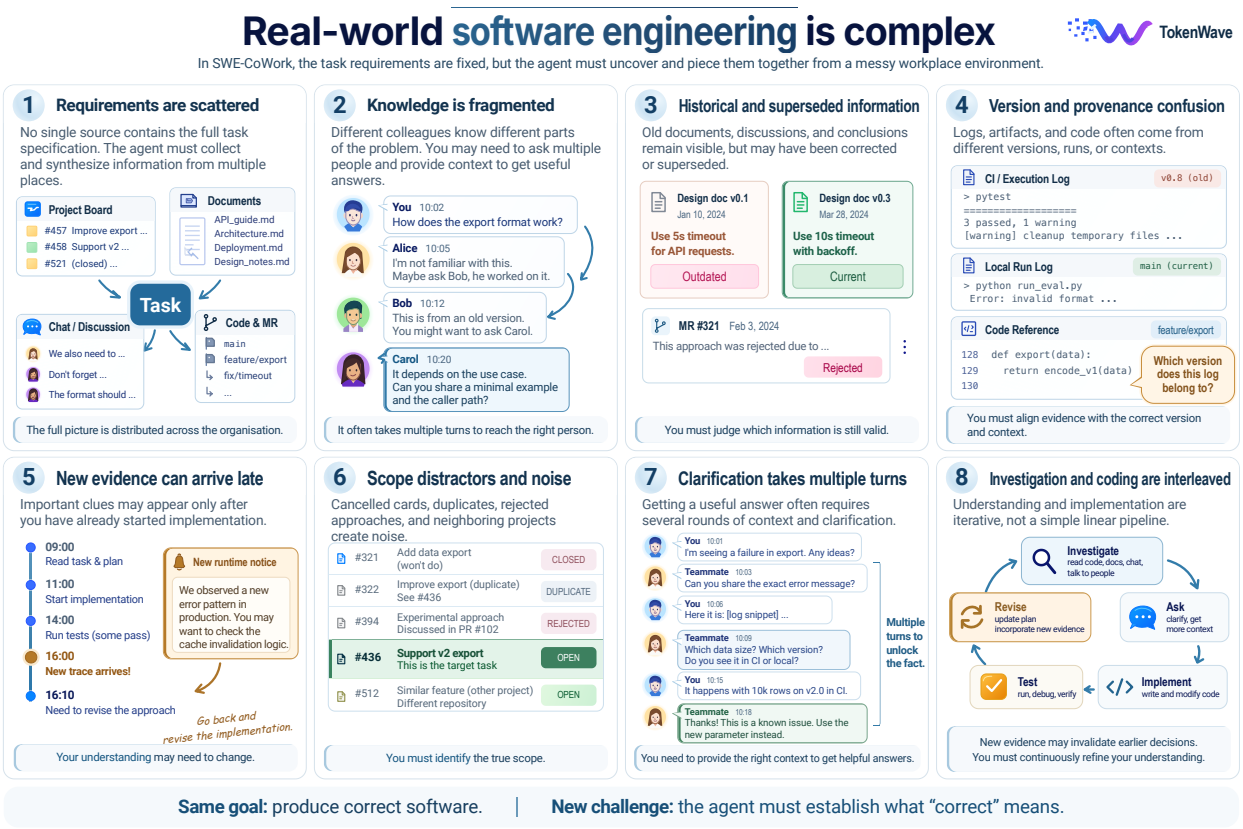


Figure 4 Eight situations represented in SWE-CoWork workplaces: scattered requirements, fragmented knowledge, superseded records, ambiguous provenance, late evidence, scope distractors, multi-turn clarification, and interleaved investigation and coding. The requirements and tests stay fixed; the agent must establish them. Dialogue and timestamps are illustrative.

wait for the prompt.

Records that need judgment. An earlier hypothesis, a successful run or a confident summary is evidence to weigh, not an instruction. Logs may be truncated, copied from an older run, or mixed with unrelated warnings; documents may carry a later retraction. Every misleading record has a visible correction or an authoritative owner, so a careful engineer can always establish which evidence applies. Appendix D shows examples from Stencil, Click and controller-runtime.

5 Evaluation

The evaluation tests the shift from a supplied coding task to work inside an organisation of employee digital twins. It has three layers: (i) an all-model CoWork evaluation, including outcomes, effort and questioning; (ii) a paired Oracle-Spec comparison for Seed 2.1 Pro; and (iii) a fine-grained trajectory analysis of Opus-5.5 as a representative strong model. The first layer measures how well each model delivers software and how it allocates effort. The second isolates the effect of supplying complete information. The third follows requirements from colleagues and records into code, verification and the final handoff.

5.1 Primary metrics

We report **F2P**, **P2P**, and **Pass rate** on $N = 101$ tasks. Let f_i and p_i be the numbers of passing F2P and P2P nodes on task i, and F_i and P_i their respective totals. All three metrics are percentages, with higher values

indicating better outcomes:

$$\text{F2P} = 100\,\frac{\sum_{i=1}^{N} f_i}{\sum_{i=1}^{N} F_i}, \qquad \text{P2P} = 100\,\frac{\sum_{i=1}^{N} p_i}{\sum_{i=1}^{N} P_i},$$

$$\text{Pass rate} = \frac{100}{N}\sum_{i=1}^{N} \mathbf{1}[f_i = F_i \ \wedge \ p_i = P_i]\,.$$

F2P measures delivery of new requirements; P2P measures preservation of protected existing behaviour. Both pool test nodes across tasks, so tasks with more nodes contribute more to the aggregate. Pass rate measures complete delivery: every F2P and every P2P node must pass in the same run. All models use the full release denominators: 2,699 F2P nodes, 562 P2P nodes, and 101 tasks. Only verifier-confirmed passes contribute to the numerators. The trajectory study uses per-task F2P fractions when relating individual runs to agent behaviour.

5.2 Experimental design

CoWork evaluation. We evaluate Opus-5.5 at max effort, Seed 2.1 Pro, DeepSeek V4 Pro, Kimi K3, GPT-6 Astra and GLM 5.3 at max effort in the CoWork setting. GPT-6 Astra is driven by a Codex scaffold; the other five models use Claude Code scaffolds. Each agent receives a brief handoff and access to the organisation, establishes the requirements through colleagues and records, and submits an implementation for the fixed F2P/P2P contract. Table 3 reports the three outcome metrics for all models on the same 101-task release contract.

Seed 2.1 Pro: Oracle-Spec comparison. The Oracle-Spec evaluation supplies Seed 2.1 Pro with the complete requirements up front. The comparison with its CoWork evaluation uses F2P, P2P and Pass rate to assess how outcomes change when the requirement set is supplied directly. Both conditions cover 101 tasks. The paired design keeps the task definitions, repository state and executable contracts fixed while changing the information supplied to the agent.

Opus-5.5: strong-model trajectory study. We analyse the CoWork trajectories of Opus-5.5 to follow the path from a colleague's knowledge to the submitted code: which requirements reach the agent, where unresolved requirements are lost, and how the agent follows up, verifies and reports its work. Opus-5.5 is selected because it is the strongest CoWork F2P performer and has the most complete trajectory archive; the study concerns this model and cohort and does not claim that its behaviour generalizes to every strong model.

5.3 Supplementary analysis metrics

Requirement acquisition and collaboration. Each task's Oracle-Spec enumerates its requirements and identifies the owner or public record that carries them. Disclosure logs and saved tool replies distinguish facts released by a colleague from replies that reached the agent. We measure **ask rate** (the share of colleague-held requirements raised with their owner or obtained), **targeting** (the share of cards whose first question reached the accountable owner), **gain per question** (disclosures per message), **follow-through** (raised requirements carried to disclosure), **redundancy** (questions about fully disclosed cards), and **unread disclosures** (replies absent from the agent's observations).

6 Results

We examine whether agents can turn an initial handoff into verified delivery. The all-model comparison reveals large differences in partial delivery but uniformly low rates of complete task success. A paired Oracle-Spec comparison then measures the burden of establishing requirements for Seed 2.1 Pro, while Opus-5.5 trajectories trace where work remains unfinished between asking, implementation, verification and reporting.

6.1 All-Model CoWork Outcomes

Table 3 compares six models on the 101-task CoWork benchmark. F2P measures delivery of new behaviours, P2P measures preservation of existing behaviours, and Pass rate requires the entire executable contract to be satisfied in the same run.

Model	F2P (%)	P2P (%)	Pass rate (%)
Opus-5.5	63.84	93.24	6.93
Seed 2.1 Pro	21.01	81.85	0.00
DeepSeek V4 Pro	13.04	90.21	0.00
Kimi K3	27.08	84.88	0.99
GPT-6 Astra	62.91	89.15	7.92
GLM 5.3	31.27	87.72	0.99

Table 3 CoWork evaluation on 101 tasks. F2P and P2P use the release totals of 2,699 and 562 nodes, respectively; Pass rate requires every F2P and P2P node to pass in the same run. All scores are percentages, with higher values indicating better outcomes.

Substantial partial delivery rarely becomes complete delivery. Opus-5.5 and GPT-6 Astra achieve the highest observed F2P scores, 63.84% and 62.91%, respectively. Yet their Pass rates are only 6.93% and 7.92%. The remaining four models achieve at most 0.99% Pass rate. Thus, the leading systems implement a substantial fraction of the requested behaviours, but none reliably delivers the complete assignment. Opus-5.5 also achieves the highest P2P score, 93.24%. Preservation alone, however, is not evidence of substantial delivery: DeepSeek V4 Pro preserves 90.21% of P2P behaviours while satisfying only 13.04% of F2P nodes. The three metrics therefore distinguish useful partial changes, regression preservation and complete delivery rather than treating them as interchangeable measures of success.

Some lower-scoring models make sparse use of colleagues. Figure 5 exposes different interaction patterns behind the lower scores. DeepSeek V4 Pro makes a median of 150 tool calls but sends 0 colleague messages per recorded run. Direct questioning is therefore absent from many of its runs, leaving the designated channel for clarifying colleague-held requirements unused. Seed 2.1 Pro and Kimi K3 also question colleagues less frequently: their median message counts are 8 and 5, with F2P scores of 21.01% and 27.08%. By comparison, Opus-5.5 and GPT-6 Astra send medians of 22 and 21 messages. These observations identify limited use of the colleague channel as a potential acquisition bottleneck; they do not establish which requirements were missed. The Oracle comparison below directly tests how much of Seed's performance can be recovered by supplying those requirements up front.

Other lower-scoring models interact extensively without comparable delivery. GLM 5.3 exhibits a different pattern: its median of 505 tool calls and 31 colleague messages exceeds both Opus-5.5 (213 calls, 22 messages) and GPT-6 Astra (178 calls, 21 messages), yet its F2P score is only 31.27%. Kimi K3 also reaches a median of 8.0 hours, the full execution budget, without approaching the leading scores. Low F2P therefore occurs both with sparse questioning and with extensive interaction; activity volume alone does not explain the outcome ordering. The relevant distinction is what the activity establishes and carries into correct code.

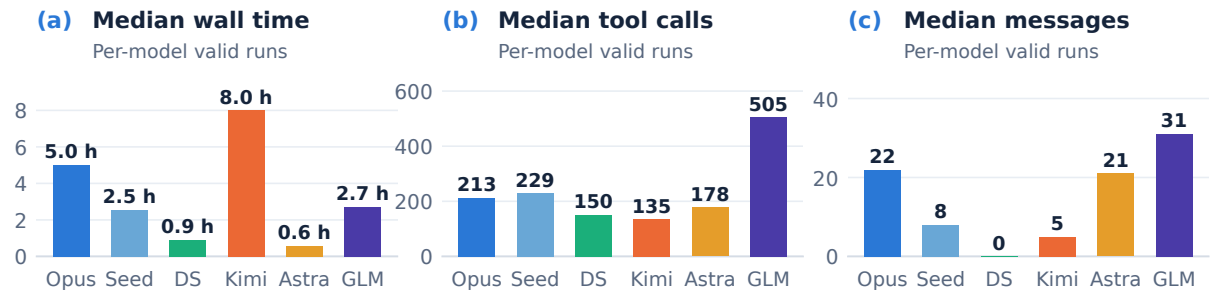


Figure 5 **Observed model effort.** Median wall-clock hours, tool calls and colleague messages per model. All models use max effort; GPT-6 Astra uses a Codex scaffold, while the other five use Claude Code.

6.2 Seed 2.1 Pro: The Cost of Establishing Requirements

The paired Oracle-Spec comparison evaluates the same model on the same 101 tasks with the complete requirements supplied up front. The repository, executable contract, scaffold and execution budget remain unchanged. This comparison asks how much of Seed 2.1 Pro's low CoWork score is recoverable when the work has already been compiled into a specification.

Metric	CoWork	Oracle	Gain (pp)
F2P	21.01	55.76	+34.75
P2P	81.85	89.50	+7.65
Pass rate	0.00	0.99	+0.99

Table 4 **Seed 2.1 Pro: CoWork vs. Oracle-Spec.** Scores are percentages; gain is in percentage points. Pooled metrics use the complete 101-task release contract.

Providing the specification raises F2P from 21.01% to 55.76%, a gain of 34.75 percentage points (Table 4). P2P also improves, from 81.85% to 89.50%, a gain of 7.65 points. The larger improvement is in delivering new behaviours: the same model implements substantially more of the executable objective once it no longer has to establish the full requirement set through the organisation. Its low CoWork score therefore cannot be explained solely by an inability to implement the underlying changes.

This gain measures the overall benefit of supplying a complete specification within a fixed budget. It includes both making requirements directly available and any search or coordination effort avoided by supplying them; it does not isolate missing knowledge from the cost of recovering it. Nevertheless, complete delivery remains rare: Pass rate increases only from 0.00% to 0.99%. Requirement recovery is therefore a substantial burden for Seed 2.1 Pro, but removing that burden is not sufficient for reliable end-to-end delivery. The agent must still translate the available specification into a complete, validated implementation.

6.3 Opus-5.5: Where End-to-End Delivery Breaks Down

We next analyse 101 CoWork trajectories from Opus-5.5, selected for its highest observed F2P score and the most complete trajectory archive. In figure 9a–c we link questions, colleague replies, code changes and delivery notes to the requirements tested by the final verifier. This identifies where requirements fail to reach the agent, where available requirements remain unsatisfied, and whether unresolved work is recognised before the agent stops. The following diagnoses concern this model and recorded cohort.

6.3.1 Accurate owner targeting does not ensure requirement coverage

Most undisclosed requirements belong to owners already contacted. The per-run mean first-question targeting rate is 93% (Table 8), indicating that questions usually reach the accountable owner on the first attempt. However, only 64% of the 2,176 colleague-held requirements are obtained (Figure 6a). For 16%, the agent raises the corresponding card with its owner but does not obtain disclosure; for 17%, it contacts

the owner about other cards without raising this one. Only 3% are held by owners never contacted. The acquisition shortfall is thus concentrated in covering the owner's relevant topics and carrying questions through to an answer, rather than simply locating a person to ask.

Obtained requirements are more often satisfied. Against the full release contract, F2P nodes linked to disclosed colleague-held requirements pass at 68.8%, compared with 49.8% for undisclosed requirements and 73.7% for requirements carried by public records (Figure 6b). Within tasks containing both disclosed and undisclosed requirements, the mean gap is 21.3 percentage points. This association is consistent with requirement acquisition contributing to implementation success.

Failure is divided between acquisition and work after receipt. Against the full release contract, Figure 6c assigns 42% of these failures to the *ask* stage: the requirement remains with a colleague. This includes 173 nodes whose card was never raised with its owner and 242 whose card was raised without obtaining disclosure. Another 56% fall in the *implement* category: the requirement reached the agent, but the tested behaviour remains wrong. The remaining 2% are assigned to *discovery*; no failed node in this attributed set is assigned to an unread disclosed reply. The *implement* category locates failure after receipt, rather than establishing that the requirement was correctly understood or isolating coding from interpretation and validation. Together, these results identify two substantial sources of unfinished work: requirements never obtained, and requirements obtained without the corresponding behaviour being delivered.

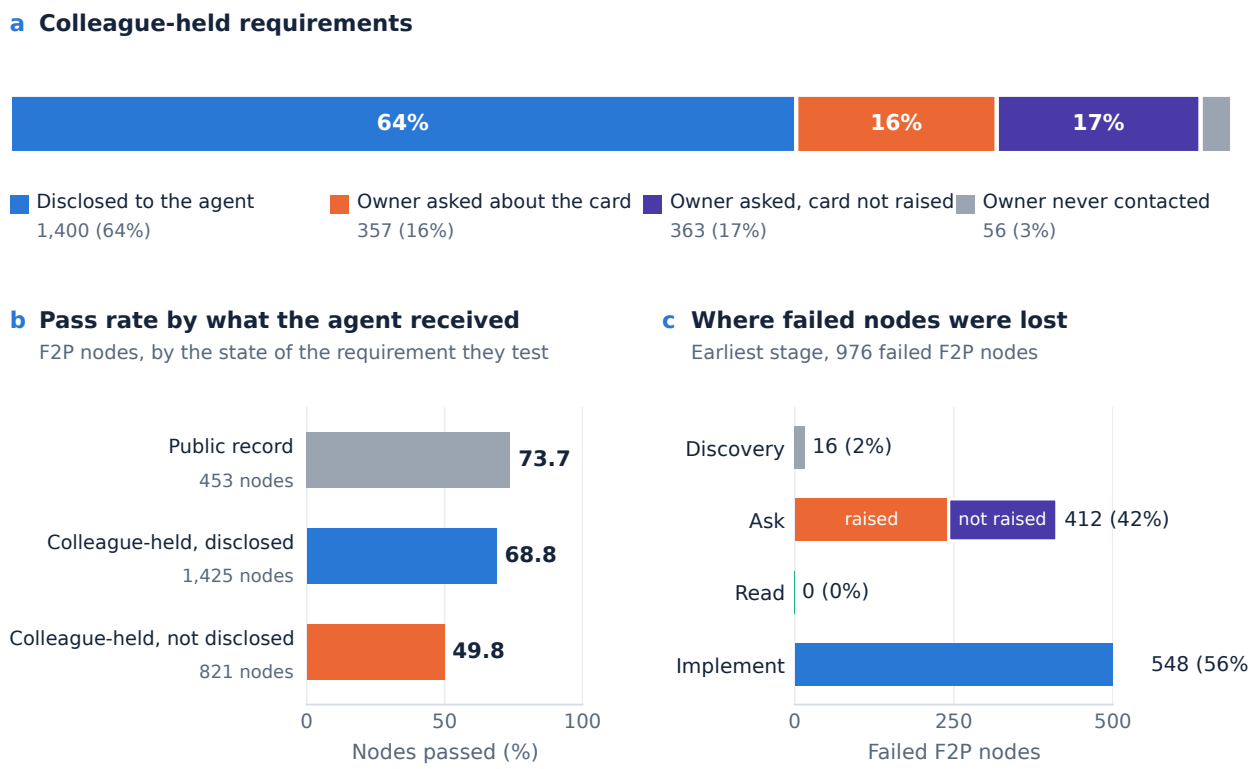


Figure 6 Requirement acquisition and loss in Opus-5.5. (a) Colleague-held requirements by acquisition state. (b) F2P node pass rate against the full release contract, by whether the corresponding requirement reached the agent. (c) Earliest stage of loss for failed nodes with linked requirements, normalized to the full release contract. The *implement* category denotes unsatisfied behaviour after the requirement reached the agent.

6.3.2 Open questions are lost while other work continues

Deferred answers expose the difference between initiating collaboration and resolving a dependency. Owners initially defer in 450 owner–card threads across 96 runs. Opus-5.5 returns to 297 threads, of which 205 eventually end in disclosure. It never follows up in the other 153 threads, and no requirement is subsequently

disclosed there (Figure 7b). At the initial deferral, the median abandoned thread still has 7.1 hours of budget left; the agent continues working for a median of 3.4 hours before stopping. These are not simply conversations opened as time expires: work continues, but the outstanding question is not resumed.

The gap is also more specific than asking only at the start. Of 2,552 message attempts, 34% occur in the first fifth of elapsed time and 20% in the final fifth (Figure 7c). Questioning therefore continues alongside implementation, while particular unanswered threads are left behind. The missing behaviour is sustained follow-through on individual questions, not merely continued use of the messaging tool.

The PostCSS task (row 014) illustrates this distinction. The agent raises card CSS-227 early, follows up with a concrete example, and then receives a second deferral after sending its own findings. It declares the streams complete while the answer is still missing. All 16 failed F2P nodes in this run concern undisclosed colleague-held requirements. The exchange contains useful investigation and follow-up, but the unresolved answer is not carried through to the completion decision.

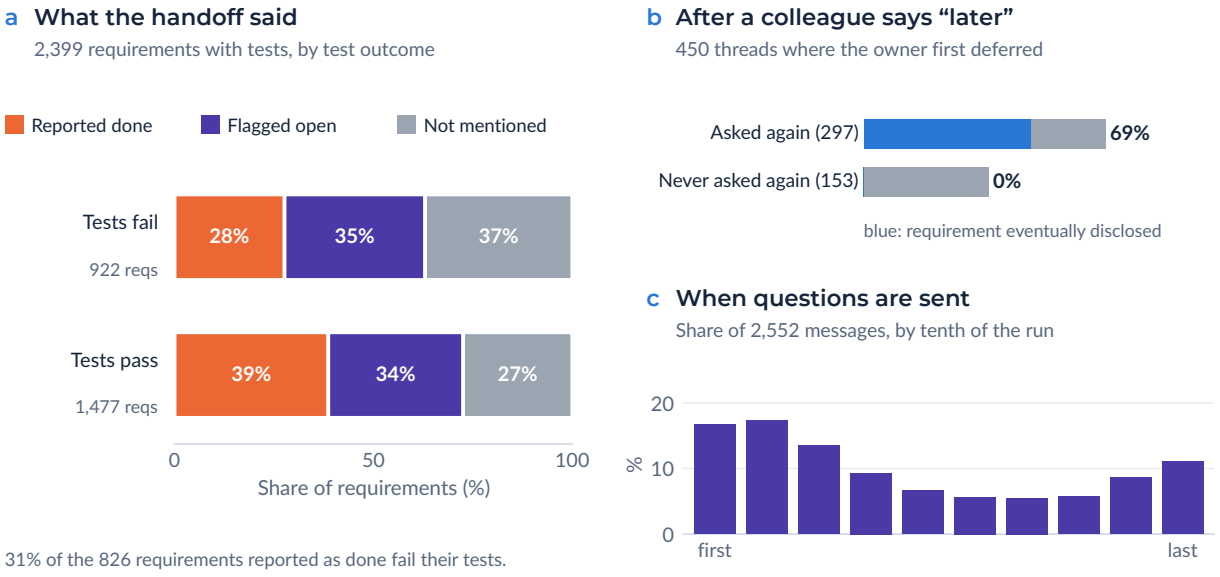


Figure 7 **Follow-through and reporting in Opus-5.5.** (a) Delivery claims compared with requirement-level test outcomes. (b) Deferred owner–card threads, follow-up and eventual disclosure. (c) Timing of colleague messages within a run. Continued questioning at the run level can coexist with individual threads receiving no follow-up.

6.3.3 Available requirements do not consistently become verified changes

The failures after receipt show why better requirement acquisition alone would not complete the workflow. In carapace-bin (row 001), Opus-5.5 obtains all 25 colleague-held requirements, yet 22/35 F2P nodes fail. The reviewer identifies mismatched usages, flags and candidate descriptions, together with an unchecked build failure. In this case, the missing step is not another disclosure: the available requirements still need to be reconciled with the exact implemented behaviour and its validation.

The exploratory behaviour profile provides a broader view of this gap (Figure 8). Higher per-task F2P is associated with more effective handling of evidence-to-code translation ($\rho = +0.36$), causal debugging and verification design ($\rho = +0.30$ each), and following owner questions through to an answer ($\rho = +0.21$). These dimensions concern what an action resolves, rather than how many actions occur. Within the same cohort, the association with raw message count is weakly positive ($\rho = +0.21$), while that with tool calls is weakly negative ($\rho = -0.21$); elapsed time has little rank association with F2P (Figure 9a–c). A median run contacts 6 colleagues and uses 5.0 hours of the eight-hour budget, but activity alone does not establish that their requirements have been implemented.

Verification-related boundaries are also not consistently preserved. Among 94 briefs that prohibit changing existing tests, 15 submitted patches change or remove existing test-source lines. This is a distinct failure to

respect the assignment's constraints, in addition to the failures to satisfy new behaviour. The traces therefore call for examining whether requirements, code changes and checks remain aligned, rather than treating the presence of coding and testing activity as evidence that the implementation has been validated.

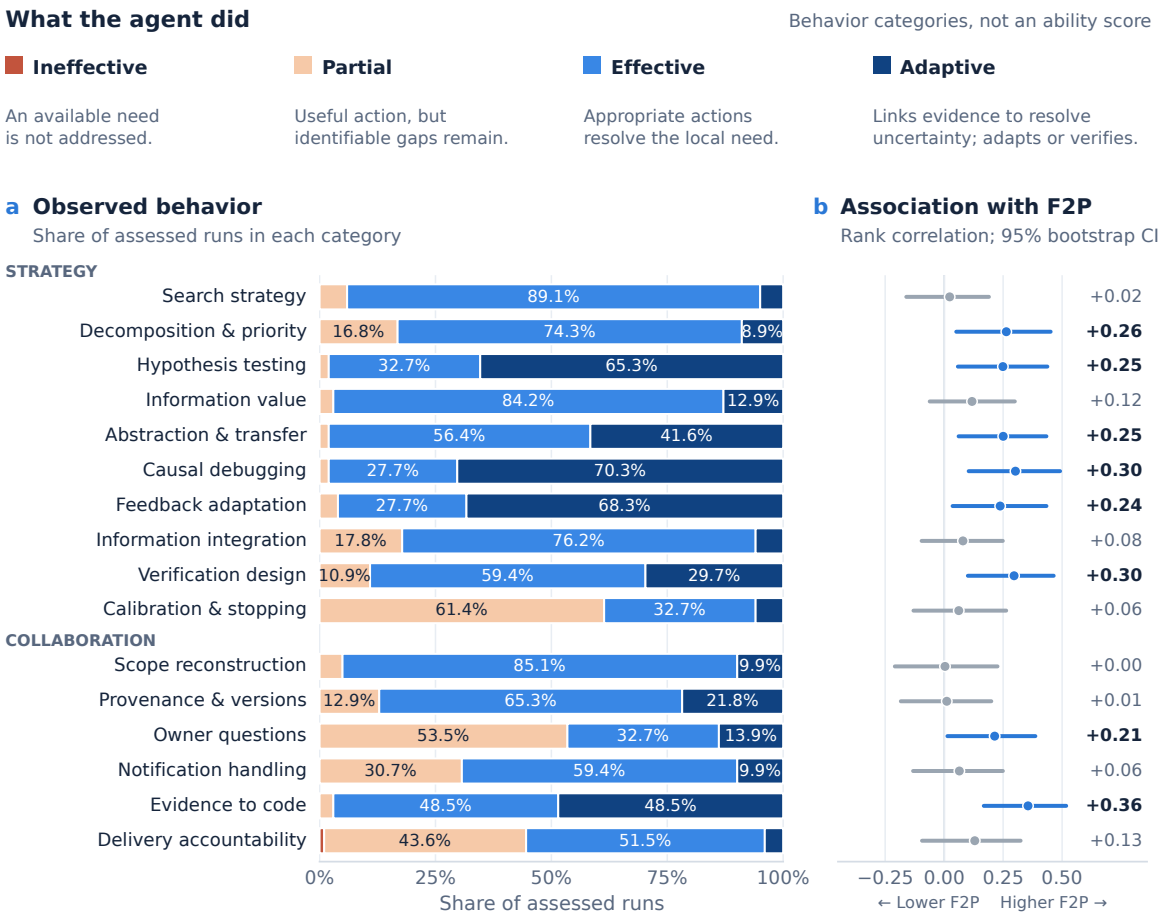


Figure 8 **Opus-5.5 behaviour profile.** (a) Percentages of assessable runs by observed behaviour: *ineffective* actions fail to address an available need; *partial* handling leaves identifiable gaps; *effective* handling resolves the local need; and *adaptive* handling links evidence across steps to resolve uncertainty and adapt or verify. Each bar is normalised over assessable cases for that dimension. Categories use dimension-specific evidence and do not imply whole-task success. (b) Spearman associations with per-task F2P, with exploratory 95% bootstrap intervals. Categories are not summed into an ability score, and associations do not establish causation.

6.3.4 Stopping and reporting are not consistently tied to unfinished work

Stopping leaves known, actionable gaps. The model reviewer classifies calibration and stopping as *partial* in 61.4% of the release cohort: useful work is visible, but identifiable gaps remain in resolving or reporting the assignment. It identifies 49 premature stops because an in-scope gap and an available next step remain (Figure 9e). Among the 48 autonomous stops in this group, the median remaining budget is 2.8 hours. Under the review protocol, these judgements are based on the ordered trajectory before private grading outcomes are joined. They therefore concern unresolved work visible in the run, rather than inferring premature stopping solely from a later hidden-test failure. The problem is not just that some tasks need more time: in the autonomous cases, a known next step remains when the agent decides to end the work.

Delivery notes obscure part of the remaining work. The claim analysis covers 2,399 requirements in 101 runs (Figure 7a). Of 922 requirements that fail their tests, handoffs flag only 35% as open, report 28% as done,

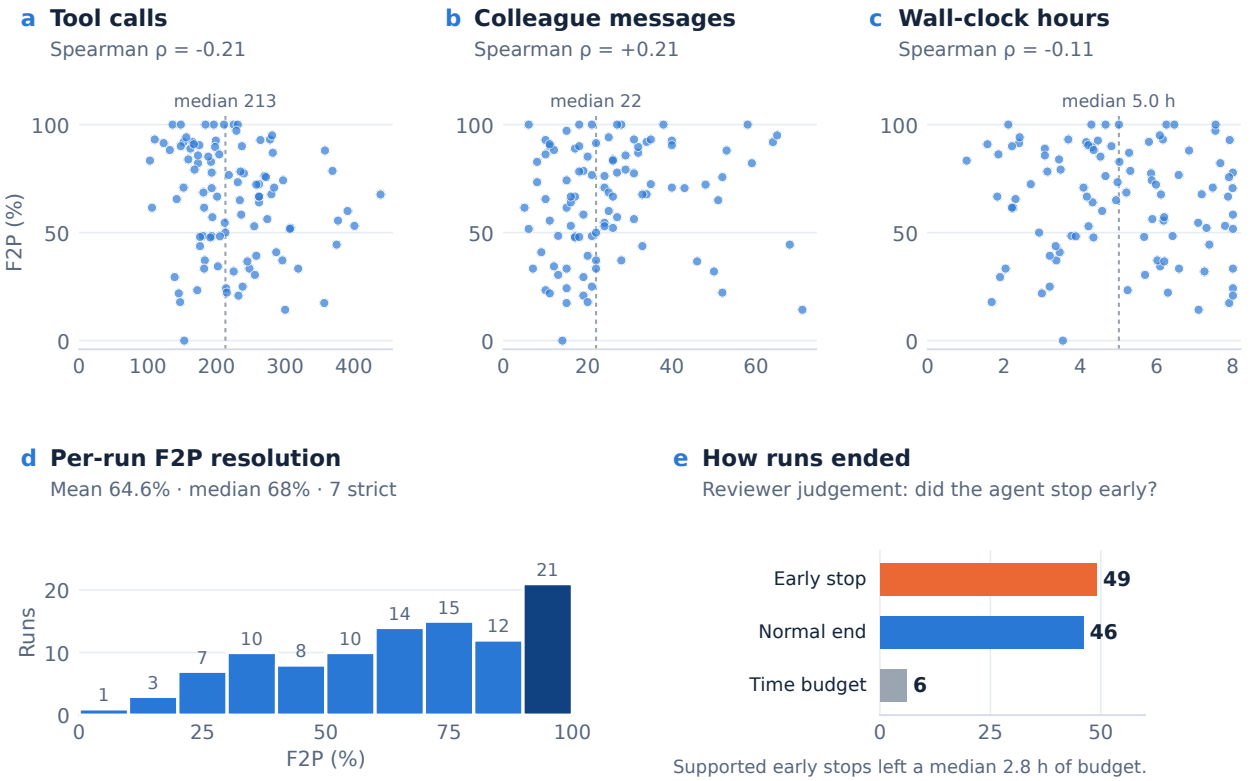


Figure 9 Effort, outcomes and stopping in Opus-5.5. (a–c) Tool calls, colleague messages and wall-clock hours against per-task F2P fractions. (d) Distribution of those fractions over the 101-task release cohort. (e) Reviewer judgements of how runs ended, including stops with known, actionable gaps.

and omit the rest. Viewed from the reader's side, 255 of the 826 requirements reported as done (31%) fail their tests. Treating all rows of "confirmed requirements" tables as completion claims changes this estimate to 382/1,157 (33%). Thus, a delivery note does not reliably distinguish satisfied requirements from unfinished ones: it both misses open work and makes completion claims contradicted by verification.

These test-based discrepancies do not by themselves establish that the agent knew the reported behaviours were wrong. HotswapAgent (row 021) provides a more direct inconsistency: the agent declares completion with 8 requirements that it has itself listed as pending. Here the completion claim conflicts with the agent's own stated work status, not only with the final verifier. This makes reporting part of the ownership problem: a team cannot reliably act on a handoff that treats acknowledged unfinished work as a completed assignment.

6.4 The Gap to Expert Engineering Practice

We use the work standard expected of a strong human engineer as a process reference, not as an evaluated human baseline. That standard does not require every assignment to succeed within a fixed budget or every hidden test to be anticipated. It requires the engineer to keep the state of the work explicit: unanswered questions remain open dependencies, implemented changes remain distinct from validated behaviour, and incomplete work is handed off as incomplete. A justified partial delivery can satisfy this standard of accountability even when it does not satisfy the full executable contract.

Against this reference, the recurring gap is continuity across the work, rather than the absence of individual engineering actions. Opus-5.5 usually routes questions to the correct owner, but does not cover all relevant topics or consistently return to deferred answers. It obtains many requirements, but does not always translate them into the required behaviour and adequate checks. It performs useful work, yet sometimes stops while an actionable gap remains and reports completion inconsistently with the remaining obligations. Each transition requires a distinction that the preceding action alone cannot settle: contacting an owner is not obtaining an

answer; receiving an answer is not satisfying its requirement; making a change is not verifying it; and ending a run is not completing the assignment.

This distinction matters for the human-led organisation envisioned in the introduction. Delegation requires both a software outcome and a dependable account of what remains. A supervisor needs to know which dependencies are still open, which behaviours have been checked, and which decisions or blockers require attention. The observed follow-through and reporting gaps limit what that supervisor can infer from an agent's completion message, even after substantial code has been produced. The strongest evaluated systems demonstrate useful partial delivery; the Opus-5.5 analysis shows why that progress is not yet consistently accompanied by verified completion or an accurate handoff of unfinished work. The remaining gap is therefore sustained, requirement-level ownership across collaboration, implementation, verification and delivery.

Appendix F reports supplementary Opus-5.5 collaboration and stratified diagnostics. Appendix G records the diagnostic definitions, claim-classification rules and evidence sources.

7 Conclusion

We propose a paradigm shift from evaluating agents on packaged coding tasks to evaluating them inside engineering organisations of employee digital twins. SWE-CoWork instantiates this setting across 101 real repositories, each with 26 modelled colleagues and a fixed executable contract. The agent must establish requirements through people and records, reconcile their evidence, and carry the work to delivery. The CoWork evaluation measures partial delivery, preservation of existing behaviour and complete task success through F2P, P2P and Pass rate. The Seed 2.1 Pro Oracle comparison examines the role of supplying requirements up front. The representative Opus-5.5 trajectory analysis identifies questions left unresolved with owners, answers implemented imprecisely, and work reported complete while its requirements still fail.

Human direction, collective agent execution. The longer-term opportunity is an organisation in which a human leads many agent programmers. The human sets goals, allocates priorities, and decides major product and technical trade-offs. The agents divide the engineering work, consult each other's expertise, coordinate interfaces and dependencies, integrate their changes, and return verified outcomes together with the decisions that need human attention. In that setting, useful autonomy means that delegated responsibility stays with the agent through uncertainty, follow-up, and delivery. Populations of employee digital twins provide a controlled setting in which to evaluate these responsibilities, beginning with an individual agent's ability to work within a team. Extending that setting to teams of autonomous programmer agents would let future evaluations ask how much work a human can reliably delegate to an entire digital workforce.

References

Cursor. CursorBench 4.0, September 2026. https://cursor.com/evals. Version 4.0, released September 10, 2026; accessed September 15, 2026.

Snagnik Das, Siddhant Paliwal, and Janak Sunil. Introducing Real-SWE. Specific Labs, September 2026. https://www.withspecific.com/benchmarks/real-swe. Private enterprise benchmark; accessed September 15, 2026.

Datacurve. DeepSWE v1.1, June 2026. https://deepswe.datacurve.ai/blog/deepswe-v1-1. Updated execution and node-id scoring release; accessed September 15, 2026.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can ai agents solve long-horizon software engineering tasks?, 2025.

Nicholas Edwards and Sebastian Schuster. Ask or assume? uncertainty-aware clarification-seeking in coding agents, 2026.

Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026.

Naman Jain. How we compare model quality in Cursor, March 2026. https://cursor.com/blog/cursorbench. March 11, 2026; accessed September 15, 2026.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Gyuhyeong Kim, Hyojung Gwon, Jeonghyeon Kim, Kyuhong Shim, and Sunjae Lee. RealSWE: A compositional evaluation of coding agents under realistic user requests, August 2026. https://arxiv.org/abs/2608.27831. Version 2, August 31, 2026.

Brendan King and Jeffrey Flanigan. Dialogue SWE-Bench: A benchmark for dialogue-driven coding agents, 2026. https://arxiv.org/abs/2606.13995.

Amy J. Ko, Robert DeLine, and Gina Venolia. Information needs in collocated software development teams. In *29th International Conference on Software Engineering (ICSE)*, pages 344–353, 2007. doi: 10.1109/ICSE.2007.45.

Rishyanth Kondra, Sanket Mhatre, Akshit Kumar, Evan Chu, Bilal Bakht Ahmad, Ayush Nangia, Rajan Agarwal, Arpan Dasgupta, Animesh Sinha, Bhuvanesh Sridharan, Krupa Dave, Brendan Graham, Guanyu Song, Anirudh Rahul, Wei Hern Lim, Abishek Thangamuthu, Ramneet Singh, Danna Liu, Navid Pour, Calvin Chen, and Justus Mattern. FrontierSWE v2. Proximal Blog, 2026. https://www.frontierswe.com/blog/v2. Version 2, 34 tasks, accessed September 15, 2026.

Thomas D. LaToza, Gina Venolia, and Robert DeLine. Maintaining mental models: A study of developer work habits. In *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, pages 492–501, 2006. doi: 10.1145/1134285.1134355.

Haobin Li, Ping Deng, Weizhong Qian, Liang Jiang, Zhenyu Huang, Mouxing Yang, and Xi Peng. Active-SWE: Benchmarking coding agents for proactive bug fixing without issue reports, 2026. https://arxiv.org/abs/2608.04682.

Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. SWE-Lancer: Can frontier LLMs earn $1 million from real-world freelance software engineering?, 2025.

OpenAI. Introducing SWE-bench verified, 2024. https://openai.com/index/introducing-swe-bench-verified/.

Zhongyuan Peng, Dan Huang, Chuyu Zhang, Caijun Xu, Changyi Xiao, Shibo Hong, David Lo, Lin Qiu, Xuezhi Cao, Jiyuan He, and Yixin Cao. ICAE-Bench: Evaluating coding agents as interactive project builders, 2026.

George Pu, Michael S. Lee, Udari Madhushani Sehwag, David J. Lee, Bryan Zhu, Yash Maurya, Mohit Raghavendra, Yuan Xue, and Samuel Marc Denton. LHAW: Controllable underspecification for long-horizon tasks, 2026.

Mohit Raghavendra, Soham Dan, Miguel Romero Calvo, Yannis Yiming He, Johannes Baptist Mols, Gautam Anand, Cole McCollum, Edgar Arakelyan, Vijay Bharadwaj, Andrew Park, Jeff Da, MohammadHossein Rezaei, Bing Liu, Brad Kenstler, and Yunzhong He. SWE Atlas: Benchmarking coding agents beyond issue resolution, 2026. https://arxiv.org/abs/2605.08366.

Preethi Seshadri, Samuel Cahyawijaya, Ayomide Odumakinde, Sameer Singh, and Seraphina Goldfarb-Tarrant. Lost in simulation: LLM-simulated users are unreliable proxies for human users in agentic evaluations, 2026.

Quan Shi, Keshav Dhandhania, Karthik Narasimhan, and Victor Barres. $\tau^{\intercal}$-bench: An environment for end-to-end, realistic agent construction, 2026. https://arxiv.org/abs/2609.04611.

Jonathan Sillito, Gail C. Murphy, and Kris De Volder. Asking and answering questions during a programming change task. *IEEE Transactions on Software Engineering*, 34(4):434–451, 2008. doi: 10.1109/TSE.2008.26.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024.

Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering*, 44(10): 951–976, 2018. doi: 10.1109/TSE.2017.2734091.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.

John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. SWE-bench multimodal: Do ai systems generalize to visual software domains? In *International Conference on Learning Representations (ICLR)*, 2025a.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents, 2025b. Introduces SWE-bench Multilingual (300 instances, 42 repositories, 9 languages).

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-SWE-bench: A multilingual benchmark for issue resolving, 2025.

Xin Zhou, Chun Yong Chong, Kisub Kim, Yun Peng, Rui Shu, Zihan Wu, Xu Han, Guowen Yuan, Zeyang Zhuang, Jounghoon Kim, Jeongjin Ju, Seongmin Ju, Taein Yoon, and David Lo. A unified issue resolution benchmark for requirement clarification, planning, and code generation for coding agents, 2026.

A Broader Benchmark Lineage

Table 5 preserves the broader conceptual comparison. The versioned comparison of recent benchmarks, with task goals, inputs and information design, appears in Table 1. SWE-bench (Jimenez et al., 2024) established repository-level patch evaluation with F2P/P2P tests. Verified (OpenAI, 2024), Multimodal (Yang et al., 2025a), Multi-SWE-bench (Zan et al., 2025) and SWE-bench Multilingual (Yang et al., 2025b) extend quality or coverage. SWE-bench Pro (Deng et al., 2025) targets larger changes, SWE-Lancer (Miserendino et al., 2025) uses paid freelance tasks, and Active-SWE (Li et al., 2026) evaluates proactive bug discovery without issue reports.

Benchmark	Focus	Information setting	Evaluation / control
SWE-bench (Jimenez et al., 2024)	Issue resolution	Issue and repository	F2P/P2P tests
SWE-bench Pro (Deng et al., 2025)	Complex software changes	Issue and repository	Repository tests
SWE-Lancer (Miserendino et al., 2025)	Paid freelance work	Task description	End-to-end tests for implementation tasks
Active-SWE (Li et al., 2026)	Proactive bug discovery and fixing	Repository without issue reports	Bug discovery and repair evaluation
TheAgentCompany (Xu et al., 2025)	Professional workflows, including SWE	Company services and multiple colleagues	Programmatic checkpoints; some model-based criteria
LHAW (Pu et al., 2026)	Controlled underspecification	Removed goals, constraints, inputs or context	Inherited evaluators and agent trials
Ask or Assume (Edwards and Schuster, 2026)	Clarification under uncertainty	Full issue held by a user simulator	Full, hidden and interactive conditions
Dialogue SWE-Bench (King and Flanigan, 2026)	Coding through dialogue	Persona-grounded user simulator	Task outcomes and dialogue quality
SWE-RPG (Zhou et al., 2026)	Requirement and planning diagnosis	Issue and repository	Executable patches and intermediate reference annotations
ICAE-Bench (Peng et al., 2026)	Interactive project building	Fuzzy requirements and a user simulator	Behavioural tests and multiple diagnostics
τ^{τ}-Bench (Shi et al., 2026)	Agent construction	Business artifacts and a client	Held-out user simulations; serving constraints
SWE-CoWork (ours)	Autonomous task completion	Distributed evidence, multiple owners and late notices	**Same-task oracle control**, F2P/P2P tests and recovery traces

Table 5 Broader conceptual lineage of repository and interactive benchmarks. Entries describe each cited setting's focus, information sources and evaluation design; they are not exclusive capability claims. The recent task-design comparison is in Table 1.

B Released Repositories

Table 6 lists the 101 upstream repositories of the 101 released tasks, one task per repository.

Domain	n	Upstream repositories
Web & Frontend	26	ada-url/ada, api-platform/core, collective/icalendar, emberjs/ember.js, feedbin/feedbin, fomantic/fomantic-ui, gioxx/marvelloussuspender, googleapis/google-api-php-client, hyperf/hyperf, ing-bank/lion, ionic-team/stencil, jasontaylordev/cleanarchitecture, jeremydaly/lambda-api, jnunemaker/httparty, jquery/jquery, kimai/kimai, mika-cn/maoxian-web-clipper, nolimits4web/swiper, python/pythondotorg, rubenlagus/telegrambots, saloonphp/saloon, shane32/qrcoder, tanstack/query, urllib3/urllib3, vacanza-/holidays, webrecorder/pywb
Dev Tools & Compilers	22	ansible/ansible-lint, carapace-sh/carapace-bin, cargo-generate/cargo-generate, cocoapods/xcodeproj, embarkstudios/cargo-deny, firebase/firebase-tools, git-lfs/git-lfs, hotswapprojects/hotswapagent, isomorphic-git/isomorphic-git, jctools/jctools, jest-community/jest-extended, junit-team/junit-examples, kpdecker/jsdiff, mdx-js/mdx, microsoft/vscode-js-debug, nedbat/coveragepy, pallets/click, postcss/postcss, psf/black, serde-rs/serde, stoplightio/elements, thought-machine/please
Data & Databases	11	alibaba/jetcache, clockworklabs/spacetimedb, dolthub/go-mysql-server, elastic/elasticsearch-rails, friendsofsymfony/foselasticabundle, kotlin-orm/ktorm, mongodb/laravel-mongodb, qishibo/anotherredisdesktopmanager, rubyzip/rubyzip, ruflin/elastica, webtorrent/webtorrent
Games & Graphics	10	caffeinemc/sodium, hubs-foundation/hubs, mezz/justenoughitems, mikke89/rmlui, multitheftauto/mtasa-blue, nasa-ammos/3dtilesrendererjs, sergio0694/computesharp, steamre/steamkit, tencent/puerts, underminer-steam/undertalemodtool
Desktop Applications	7	betterdiscord/betterdiscord, bobotig/python-mss, electronnet/electron.net, ipfs/ipfs-desktop, openutau/openutau, pimutils/khal, sozi-projects/sozi
Cloud & Infrastructure	5	boto/boto3, brightercommand/brighter, kubermatic/kubeone, robusta-dev/robusta, stackstorm/st2
Finance & Payments	5	btcsuite/btcd, ledgerhq/ledger-live, mempool/mempool, stripe/stripe-php, stripe/stripe-ruby
ML & AI Systems	4	huggingface/transformers.js, infiniflow/ragflow, lmcache/lmcache, onnx/onnx
Security & Identity	4	apache/caldera, bubka/2fauth, crytic/slither, pglombardo/passwordpusher
Mobile Applications	3	luoyesiqiu/dpt-shell, mygod/vpnhotspot, react-native-async-storage/async-storage
Observability	2	errbit/errbit, newrelic/newrelic-ruby-agent
Scientific Computing	2	angularsen/unitsnet, google-deepmind/mujoco

Table 6 The 101 released tasks by application domain and upstream repository.

C Evidence Arriving Mid-Build

An agent finds a successful browser record and concludes that the consumer path is covered. During implementation a new message reports a contrasting SDK observation and asks for the client version. The agent takes the lead to the owner, supplies the request and chunking context, and learns the behaviour that applies to the second path. The browser record was right for its own setting; the mistake would have been to generalise it. Owners can be asked at any time, so an agent that asks early has the answer before the message arrives.

D Evidence Case Examples

Figure 10 shows the historical, provenance and scope judgments of Section 4.3 in workplace records from Stencil, Click and a controller-runtime development snapshot. The Click records also underlie the opening comparison in Figure 1, where the issue isolates the output-capture requirement from `DOC-11`.

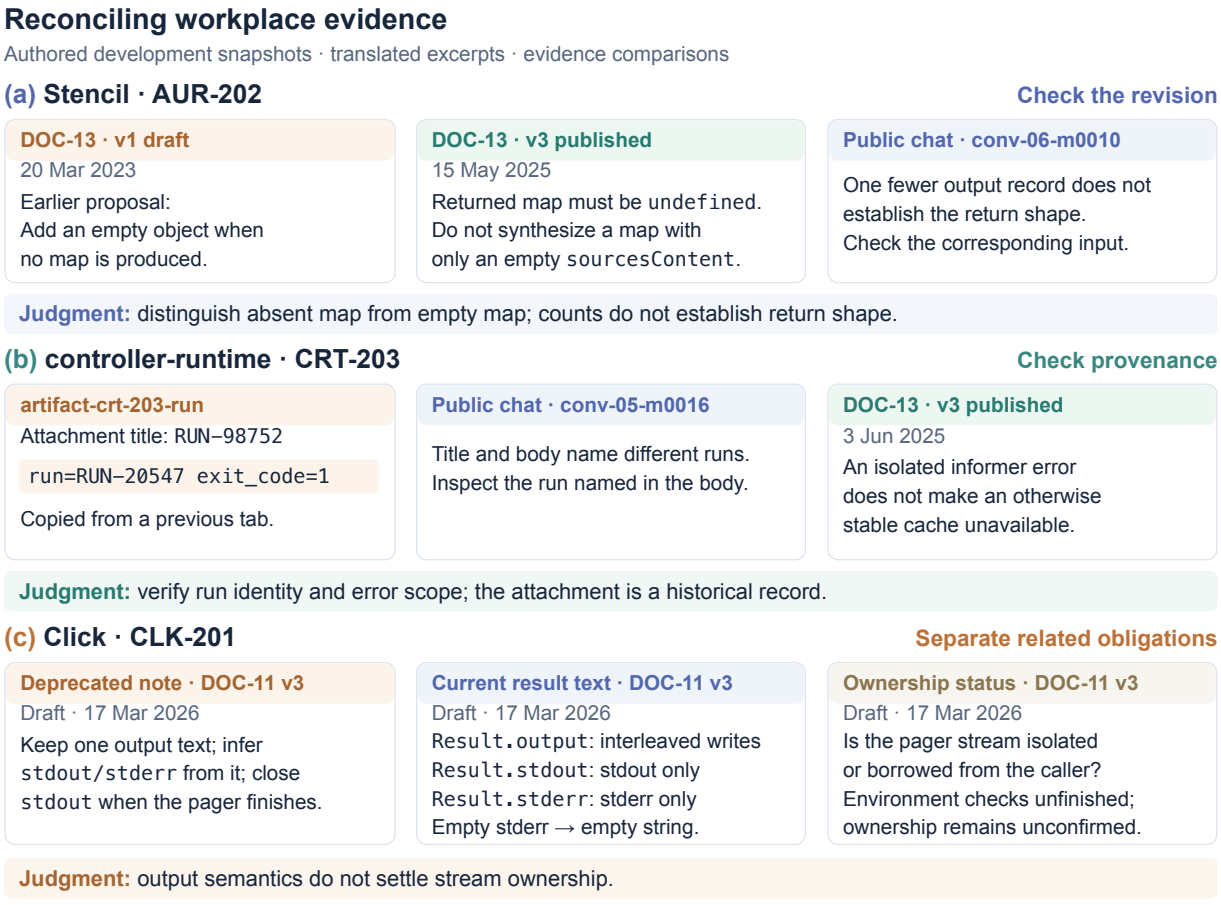


Figure 10 Evidence judgments in workplace records, with source card, document, version and message identifiers. (a) A superseded return shape versus the published correction. (b) A copied run excerpt that needs an identity check. (c) Result semantics separated from an open resource-ownership question.

Stencil: an absent value is not an empty object. Card `AUR-202` leads to `DOC-13`, where the published correction supersedes an earlier proposal to fill in an empty map. The chunk must retain its other output information even when `map` is `undefined`. The linked run excerpt reports 22 versus 21 output records, without exposing their shapes. The public reply asks for the corresponding input: aggregate counts cannot distinguish an absent map from an empty object.

controller-runtime: run identity and error scope are separate checks. Around `CRT-203`, a public message identifies a mismatch between the attachment title and the run named inside copied terminal output. The linked `DOC-13` also corrects an earlier cache-wide interpretation of an informer error. These checks answer different questions: which execution produced an observation, and whether that observation implies loss of an otherwise stable cache. An attachment title or failing exit code alone resolves neither.

Click: output observations do not determine stream ownership. The handoff on `CLK-201` points to `DOC-11`. Its third version, marked draft, distinguishes interleaved output from source-specific output while retaining a deprecated simplification. It also leaves the pager's stream ownership unresolved after an incomplete

environment check. Knowing the result fields does not establish who supplied the actual stream or who still uses it after return. The available answer must therefore remain separate from the adjacent, unanswered lifecycle question.

E Per-Task Statistics Table

Table 7 summarises software, workplace and information-design quantities across the released tasks.

Quantity per task	Mean	Median	P25	P75	Max
Code task					
Verifier test nodes	32.3	32	28	35	68
Fail-to-pass test nodes	26.7	27	23	31	67
Pass-to-pass test nodes	5.6	4	3	8	19
Production files in golden patch	97.5	60	37	133	524
Production lines added	4,590.3	2,955	1,687	5,645	26,605
Production lines deleted	1,752.0	1,069	468	2,217	13,232
Workplace					
Colleagues	26.0	26	26	26	26
Board cards	102.3	102	100	104	107
Card comments	62.2	63	58	66	80
Documents	50.5	50	48	52	56
Chat messages	404.3	405	399	410	417
Merge requests	11.8	12	11	13	14
MR review notes	27.8	28	26	30	35
Run artifacts	49.9	50	47	53	64
Information design					
Scored behavioural facts	26.9	28	24	30	32
Contract atoms pinned by tests	28.6	28	18	34	88
Static contract atoms (D_0)	11.1	5	5	14	58
Facts held by an owner (Q)	22.1	23	19	25	28
Progress-gated deliveries (D)	8.0	8	8	8	8

Table 7 Per-task statistics over all 101 tasks. Production lines and files exclude tests, documentation, configuration, generated files, fixtures, lockfiles and vendored code.

F Supplementary Opus-5.5 Diagnostics

The following tables support the representative trajectory study in Section 6.3. They describe acquisition behaviour and outcomes across strata within the Opus-5.5 trajectory archive.

	Ask rate	Targeting	Gain/Q	Follow-thr.	Redundancy	Unread
Opus-5.5 (CoWork)	80.4	93.3	0.64	78.3	13.7	0.2
Spearman ρ with F2P	+0.11	+0.14	+0.04	+0.13	+0.14	+0.18

Table 8 **Opus-5.5 acquisition behaviour.** The first row contains per-run means; the second contains their Spearman associations with per-task F2P. Ask rate, targeting, gain per question, follow-through, redundancy and unread disclosures follow Section 5. All means are percentages except gain per question.

Axis	Stratum	n	F2P (%)	Passed tasks	Disclosed (%)
Verifier nodes	< 28	21	70.1	3	76.5
	28–35	55	62.5	2	61.3
	> 35	25	64.7	2	61.9
Edit size	< 1k production lines	6	67.6	0	75.0
	1k–5k	58	66.0	5	65.4
	> 5k	37	62.0	2	61.7
Language	Python	17	69.3	1	62.2
	TypeScript / JavaScript	32	57.3	1	64.1
	PHP / Ruby	19	68.4	1	68.2
	Go / Rust	11	56.5	0	72.6
	C++ / C# / Java / Kotlin	22	72.3	4	60.2
Domain	Web & frontend	26	75.8	4	72.2
	Dev tools & compilers	22	63.9	2	61.8
	Data & databases	11	70.2	1	67.6
	Games & graphics	10	48.9	0	58.5
	Other domains	32	58.9	0	61.4

Table 9 Stratified Opus-5.5 diagnostics. Here F2P is the mean of per-task fractions within each stratum, rather than the pooled outcome score in Table 3. Passed tasks counts joint F2P/P2P successes; Disclosed is the mean share of colleague-held requirements obtained. Edit size counts production lines added by the reference change.

G Opus-5.5 Trajectories: Measurement Details

Trajectory archive. The analysis is framed by the 101-task release cohort; the available Opus-5.5 CoWork archive supplies the behavioural measurements, while Table 3 reports benchmark outcomes pooled against the complete test contract. Each record carries its task version, package hash, run ID, requested models, scaffold version, runtime revision and budget. A trajectory that resumes after a provider interruption remains a single observation.

Linking tests to requirements. For each F2P node represented in the trajectory archive, we identify the requirements it tests by matching per-node requirement files, test sources and upstream change titles against the Oracle-Spec of the same package.

Operational loss stages. The attribution uses card identifiers, document-retrieval metadata and disclosure events to locate observable breaks in the information flow.

Disclosure and visibility. The workplace service records every requirement it releases to the agent as a committed disclosure, linked to the question that triggered it. A disclosure counts as reaching the agent when the reply body appears in a tool result of the saved trajectory. Questions are linked to cards by the card identifiers they name. A requirement whose card the agent never named, but whose owner it did contact, is counted as *owner contacted, card not raised*.

Behaviour annotation. The reviewer applies ten strategy and six collaboration dimensions to observable actions and their results. The four ordered categories are *ineffective* (an available need is not addressed), *partial* (useful action leaves identifiable gaps), *effective* (appropriate actions resolve the local need) and *adaptive* (evidence links multiple steps to resolve uncertainty and adapt or verify). The four categories are encoded as 0–3 for rank-based analysis; these codes are not an ability score. A category is assigned only when the available evidence supports a dimension-specific judgement. Insufficient evidence or lack of an opportunity to exhibit the behaviour is marked *not assessable*; an absent or incomplete dimension review is recorded as missing. Neither is assigned to one of the four behaviour categories.

Category percentages are computed over assessable runs separately for each dimension. Spearman correlations additionally require an observed per-task F2P score. A run without an assessment for one dimension may still contribute to other dimensions and diagnostics for which the required records are available. For owner-question follow-through, for example, a relevant question without necessary follow-up can be partial; carrying a question

through to an actionable answer can be effective; resolving a consequential ambiguity through linked questions and corroborating evidence can be adaptive.

Dimension	Assessed n	I	P	E	A
Search strategy	101	0	6	90	5
Decomposition/prioritization	101	0	17	75	9
Hypothesis testing	101	0	2	33	66
Information value	101	0	3	85	13
Abstraction/transfer	101	0	2	57	42
Causal debugging	101	0	2	28	71
Feedback adaptation	101	0	4	28	69
Information integration	101	0	18	77	6
Verification design	101	0	11	60	30
Calibration/stopping	101	0	62	33	6
Scope reconstruction	101	0	5	86	10
Provenance/version reconciliation	101	0	13	66	22
Owner questions/follow-through	101	0	54	33	14
Notification handling	101	0	31	60	10
Evidence to implementation	101	0	3	49	49
Delivery accountability	101	1	44	52	4

Table 10 Counts of behavior categories for the trajectories. I: ineffective; P: partial; E: effective; A: adaptive. Counts are reported over the 101-task release cohort. Definitions are given above. Each dimension has distinct behavioral anchors; categories are not summed across dimensions. Records without a complete dimension review are treated as *partial* for cohort presentation.

Delivery claims. For each requirement we located its card identifier, or its topic when the card is not named, in the delivery note and in the agent's final completion summary, and classified the surrounding sentence, table row or section as reporting the requirement done, open (pending, unconfirmed, partial, an own assumption, or listed under an open-items heading) or not mentioned. Topic matches under an open-items heading count as open even when the card is named elsewhere, and a requirement listed as open in any of these ways counts as open. Wide card ranges in source lists count as a claim only when the sentence quantifies over all of them. The main figures treat rows of "confirmed requirements" tables as claims only when they also state an implementation; the broader reading is reported alongside.

Deferrals and rules. A deferred thread is an owner–card pair whose first reply deferred the answer; the agent follows up when it later messages the same owner naming the same card, or when the environment counts a qualifying follow-up for that thread. Budget left uses the eight-hour budget and elapsed time since the first saved action; subsequent working time ends at the final saved action. Test-rule compliance counts changed or removed lines in existing test source files of the submitted patch, for runs whose brief states the test-preservation rule without a migration exception.